SAVEETHA SCHOOL OF ENGINEERING

SIMATS UNIVERSITY

Name: Hitesh C

Register No: 192524005

Course Code: CSA1401

Course Name: COMPILER DESIGN

Faculty Name: Dr. V. AJITHA

QUESTION 1

```
#include <stdio.h>
int sumArray(int arr[], int n)
{
   int i, sum = 0;
   for(i = 0; i < n; i++)
   {
      sum = sum + arr[i];
   }
   return sum;
}
int main()
{
   int arr[5] = {1,2,3,4,5};
   int total;
   total = sumArray(arr, 5);
   if(total == 15)
```

```
    {
        printf("Correct Sum\n");
    }
    int a = 10;
    int b = 2;
    int c = a / b;
    printf("Division Result = %d\n", c);
    int value = 20;
    int *p = &value;
    printf("Pointer Value = %d\n", *p);
    return 0;
}
```

a) Error Identification

Errors:

- Missing semicolon after array initialization.
- Loop condition uses i<=n instead of i<n.
- sumArray() is called without passing n.
- Missing semicolon after printf().
- Division by zero.
- Uninitialized pointer dereference.

These errors prevent correct compilation or lead to runtime failure.

b) Classification of Errors

Syntax Errors: Missing semicolons and braces.
Semantic Error: Incorrect function call.
Logical Error: Wrong loop condition.
Runtime Errors: Division by zero and invalid pointer access.

c) Compilation Behaviour

Lexical, syntax and semantic errors are detected by the compiler. Runtime errors such as division by zero occur only during execution. Early syntax errors may stop the compiler from analyzing the remaining program.

d) Corrected Code

```
int total = sumArray(arr,5);
for(i=0;i<n;i++)
   sum += arr[i];

if(total==15){
   printf("Correct Sum\n");
}

int b=2;
int c=a/b;
int value=20;
int *p=&value;
*p=20;
```

e) Impact Analysis

Improper pointer usage may crash the program. Division by zero terminates execution. Incorrect loop conditions may access memory outside the array and produce undefined behaviour.

f) Compiler Improvements

Compilers can provide better error messages, static analysis, runtime sanitizers and intelligent debugging suggestions.

QUESTION 2

```
#include <stdio.h>

int main()

{

   int num1 = 100;

   float rate = 25.5;

   char ch = 'a';

   int total = 50 + 20;

   int num1_copy = 30;

   int count = 40;

   printf("Welcome to Compiler Design\n");
```

```
    int x = 9;
    int y = 0x12;
    float z = 12.34;
    char str1[] = "Hello";
    char str2[] = "World";
    int _valid = 10;
    int invalid_id = 20;
    float value = 5.6;
    int data = 10 + 5;
    char c = 'x';
    int value3 = 60;
    printf("Value: %d\n", total);
    return 0;
}
```

a) Error Identification

Invalid tokens include 1num, rate@, 50$20, num#1, %count, 09, 0xZ12, 12.3.4, invalid-id, 5..6, 10@5, 'xy', 3value and unterminated strings.

b) Error Classification

These violate identifier formation, constant rules, string literal rules and operator/symbol rules.

c) Token Correction

1num→num1
rate@→rate
num#1→num1
09→9
0xZ12→0x12
12.3.4→12.34
'xy'→'x'

d) Compiler Behaviour

The lexical analyzer scans characters, forms tokens and reports invalid tokens. It may continue scanning using error recovery.

e) DFA Rule

An identifier must begin with a letter or underscore. Remaining characters may be letters, digits or underscores.

QUESTION 3

```
#include <stdio.h>

int divide(int a, int b)

{

   if(b == 0)

   {

      printf("Division by zero is not allowed.\n");

      return 0;

   }

   return a / b;

}

int main()

{

   int value = 20;

   float num = 15.5;

   int result;

   result = divide(10, 2);

   if(result == 5)

   {

      printf("Result is 5\n");

   }

   int x = 10;
```

```
    int y = 2;
    int z = x / y;
    printf("Division Result = %d\n", z);
    int arr[4] = {1,2,3,4};
    printf("First Element = %d\n", arr[0]);
    int data = 25;
    int *ptr = &data;
    *ptr = 25;
    printf("Pointer Value = %d\n", *ptr);
    int total;
    total = x + y * result;
    printf("Total = %d\n", total);
    while(x < 20)
    {
        printf("%d\n", x);
        x++;
    }
    return 0;
}
```

a) Identify Errors

Errors include missing semicolons, invalid identifier #value, wrong function arguments, assignment instead of comparison, division by zero, array index out of bounds and invalid pointer.

b) Classification

Lexical, Syntax, Semantic, Intermediate Code, Optimization and Runtime.

c) Compilation vs Execution

Compile-time errors stop translation whereas runtime errors appear only when executing the program.

d) Corrected Solution

Add missing semicolons, pass correct arguments, replace '=' with '==', initialize pointers, remove redundant '+0' and avoid division by zero.

e) Deep Analysis

Invalid pointers and array overflow may corrupt memory. Operator precedence affects intermediate code generation.

f) Advanced Task

Use static analysis, bounds checking, optimization and improved diagnostics for better debugging.